🎓 Phase 7: Practice Challenges

Challenge 1: Inventory Management

Write queries to:

1. Identify products that need reordering across all warehouses

```
WITH total_stock AS (
  SELECT
    i.product_id,
    COALESCE(SUM(i.quantity), 0) AS total_qty_all_warehouses
  FROM inventory i
  GROUP BY i.product_id
)
SELECT
  p.product_id,
  p.product_name,
  p.reorder_level,
  ts.total_qty_all_warehouses,
  (p.reorder_level - ts.total_qty_all_warehouses) AS units_to_order
FROM products p
JOIN total_stock ts ON ts.product_id = p.product_id
WHERE ts.total_qty_all_warehouses < p.reorder_level
ORDER BY units_to_order DESC, p.product_name;
```

2. Calculate the cost of restocking all products below reorder level

```
WITH total_stock AS (
  SELECT
    i.product_id,
    COALESCE(SUM(i.quantity), 0) AS total_qty_all_warehouses
  FROM inventory i
  GROUP BY i.product_id
),
```

```
needs AS (
  SELECT
    p.product_id,
    p.product_name,
    p.cost_price,
    p.reorder_level,
    ts.total_qty_all_warehouses,
    GREATEST(p.reorder_level - ts.total_qty_all_warehouses, 0) AS units_to_order
  FROM products p
  JOIN total_stock ts ON ts.product_id = p.product_id
  WHERE ts.total_qty_all_warehouses < p.reorder_level
)
SELECT
  product_id,
  product_name,
  reorder_level,
  total_qty_all_warehouses,
  units_to_order,
  cost_price,
  ROUND(units_to_order * cost_price, 2) AS restock_cost,
  ROUND(SUM(units_to_order * cost_price) OVER (), 2) AS grand_total_restock_cost
FROM needs
ORDER BY restock_cost DESC, product_name;
```

3. Recommend warehouse transfers to balance inventory

```
WITH wh_count AS (
  SELECT COUNT(*)::int AS n FROM warehouses
),
prod_totals AS (
  SELECT i.product_id, SUM(i.quantity)::int AS total_qty
  FROM inventory i
  GROUP BY i.product_id
),
targets AS (
  SELECT
    pt.product_id,
    pt.total_qty,
    CEIL(pt.total_qty::numeric / NULLIF(wc.n, 0))::int AS target_per_wh
  FROM prod_totals pt
```

```
  CROSS JOIN wh_count wc
),
inv AS (
  SELECT
    i.product_id,
    i.warehouse_id,
    i.quantity::int AS qty,
    t.target_per_wh
  FROM inventory i
  JOIN targets t ON t.product_id = i.product_id
),
surplus AS (
  SELECT product_id, warehouse_id, (qty - target_per_wh)::int AS surplus_qty
  FROM inv
  WHERE qty > target_per_wh
),
deficit AS (
  SELECT product_id, warehouse_id, (target_per_wh - qty)::int AS deficit_qty
  FROM inv
  WHERE qty < target_per_wh
)
SELECT
  p.product_id,
  p.product_name,
  w_from.warehouse_name AS from_warehouse,
  w_to.warehouse_name   AS to_warehouse,
  LEAST(s.surplus_qty, d.deficit_qty) AS suggested_transfer_qty
FROM surplus s
JOIN deficit d
  ON d.product_id = s.product_id
AND d.warehouse_id <> s.warehouse_id
JOIN products p ON p.product_id = s.product_id
JOIN warehouses w_from ON w_from.warehouse_id = s.warehouse_id
JOIN warehouses w_to   ON w_to.warehouse_id   = d.warehouse_id
WHERE LEAST(s.surplus_qty, d.deficit_qty) > 0
ORDER BY p.product_name, suggested_transfer_qty DESC;
```

Challenge 2: Customer Analytics WITH cohorts AS (

1. Create a customer cohort analysis by registration month

```
SELECT
    c.customer_id,
    date_trunc('month', c.registration_date)::date AS cohort_month
  FROM customers c
),
cohort_sizes AS (
  SELECT cohort_month, COUNT(*) AS cohort_size
  FROM cohorts
  GROUP BY cohort_month
),
activity AS (
  SELECT
    co.cohort_month,
    date_trunc('month', o.order_date)::date AS order_month,
    COUNT(DISTINCT o.customer_id) AS active_customers
  FROM cohorts co
  JOIN orders o ON o.customer_id = co.customer_id
  WHERE o.order_status <> 'Cancelled'
  GROUP BY co.cohort_month, date_trunc('month', o.order_date)::date
)
SELECT
  a.cohort_month,
  a.order_month,
  cs.cohort_size,
  a.active_customers,
  ROUND(a.active_customers * 100.0 / NULLIF(cs.cohort_size, 0), 2) AS retention_rate_pct
FROM activity a
JOIN cohort_sizes cs USING (cohort_month)
ORDER BY cohort_month, order_month;
```

2. Calculate customer churn rate

```
WITH last_orders AS (
  SELECT
    c.customer_id,
    MAX(o.order_date)::date AS last_order_date
  FROM customers c
  LEFT JOIN orders o
    ON o.customer_id = c.customer_id
   AND o.order_status <> 'Cancelled'
  GROUP BY c.customer_id
)
SELECT
  $1::int AS churn_days,
  COUNT(*) AS total_customers,
  SUM(
    CASE
      WHEN last_order_date IS NULL THEN 1
      WHEN last_order_date < (CURRENT_DATE - ($1::int * INTERVAL '1 day')) THEN 1
      ELSE 0
    END
  ) AS churned_customers,
  ROUND(
    SUM(
      CASE
        WHEN last_order_date IS NULL THEN 1
        WHEN last_order_date < (CURRENT_DATE - ($1::int * INTERVAL '1 day')) THEN 1
        ELSE 0
      END
    ) * 100.0 / NULLIF(COUNT(*), 0),
    2
  ) AS churn_rate_pct
FROM last_orders;
```

3. Identify customers likely to upgrade loyalty tiers

```
WITH thresholds AS (
  SELECT
    c.customer_id,
    c.first_name,
    c.last_name,
    c.email,
    c.loyalty_tier,
    c.total_spent,
    CASE c.loyalty_tier
      WHEN 'Bronze' THEN 1000
      WHEN 'Silver' THEN 5000
      WHEN 'Gold' THEN 10000
      ELSE NULL
    END AS next_threshold,
    CASE c.loyalty_tier
      WHEN 'Bronze' THEN 'Silver'
      WHEN 'Silver' THEN 'Gold'
      WHEN 'Gold' THEN 'Platinum'
      ELSE NULL
    END AS next_tier
  FROM customers c
),
last_order AS (
  SELECT customer_id, MAX(order_date)::date AS last_order_date
  FROM orders
  WHERE order_status <> 'Cancelled'
  GROUP BY customer_id
)
SELECT
  t.customer_id,
  (t.first_name || ' ' || t.last_name) AS customer_name,
  t.email,
  t.loyalty_tier,
  t.next_tier,
  t.total_spent,
  t.next_threshold,
  ROUND((t.next_threshold - t.total_spent), 2) AS amount_needed,
  lo.last_order_date
FROM thresholds t
LEFT JOIN last_order lo ON lo.customer_id = t.customer_id
WHERE t.next_threshold IS NOT NULL
```

```
  AND (t.next_threshold - t.total_spent) > 0
  AND (t.next_threshold - t.total_spent) <= (t.next_threshold * $1::numeric)
  AND (lo.last_order_date IS NULL OR lo.last_order_date >= CURRENT_DATE - INTERVAL '90 days')
ORDER BY amount_needed ASC, t.total_spent DESC;
```

Challenge 3: Revenue Optimization

1. Find the most profitable product combinations

```
WITH lines AS (
  SELECT
    oi.order_id,
    oi.product_id,
    oi.quantity,
    oi.subtotal,
    p.cost_price,
    (oi.subtotal - (p.cost_price * oi.quantity))::numeric(12,2) AS line_profit
  FROM order_items oi
  JOIN products p ON p.product_id = oi.product_id
  JOIN orders o ON o.order_id = oi.order_id
  WHERE o.order_status <> 'Cancelled'
),
pairs AS (
  SELECT
    l1.product_id AS product_a_id,
    l2.product_id AS product_b_id,
    COUNT(*) AS times_bought_together,
    SUM(l1.line_profit + l2.line_profit)::numeric(12,2) AS pair_profit
  FROM lines l1
  JOIN lines l2
    ON l2.order_id = l1.order_id
   AND l2.product_id > l1.product_id
  GROUP BY l1.product_id, l2.product_id
)
SELECT
  pa.product_name AS product_a,
  pb.product_name AS product_b,
  times_bought_together,
```

```
  pair_profit
FROM pairs
JOIN products pa ON pa.product_id = pairs.product_a_id
JOIN products pb ON pb.product_id = pairs.product_b_id
ORDER BY pair_profit DESC, times_bought_together DESC
LIMIT $1::int;
```

2. Analyze discount effectiveness on revenue

```
SELECT
  p.product_id,
  p.product_name,
  SUM(CASE WHEN oi.discount > 0 THEN oi.quantity ELSE 0 END) AS discounted_units,
  SUM(CASE WHEN oi.discount = 0 THEN oi.quantity ELSE 0 END) AS fullprice_units,
  ROUND(SUM(CASE WHEN oi.discount > 0 THEN oi.subtotal ELSE 0 END), 2) AS
discounted_revenue,
  ROUND(SUM(CASE WHEN oi.discount = 0 THEN oi.subtotal ELSE 0 END), 2) AS fullprice_revenue,
  ROUND(
    AVG(
      CASE
        WHEN oi.discount > 0
        THEN (oi.discount / NULLIF(oi.unit_price * oi.quantity, 0)) * 100
      END
    ),
    2
  ) AS avg_discount_pct
FROM order_items oi
JOIN products p ON p.product_id = oi.product_id
JOIN orders o ON o.order_id = oi.order_id
WHERE o.order_status <> 'Cancelled'
GROUP BY p.product_id, p.product_name
ORDER BY discounted_revenue DESC NULLS LAST, fullprice_revenue DESC;
```

3. Calculate revenue per warehouse

```
WITH latest_shipment AS (
  SELECT DISTINCT ON (s.order_id)
    s.order_id,
    s.warehouse_id,
    s.shipment_date
  FROM shipments s
  ORDER BY s.order_id, s.shipment_date DESC
)
SELECT
  w.warehouse_id,
  w.warehouse_name,
  COUNT(DISTINCT o.order_id) AS orders_count,
  ROUND(SUM(o.total_amount), 2) AS revenue
FROM latest_shipment ls
JOIN orders o ON o.order_id = ls.order_id
JOIN warehouses w ON w.warehouse_id = ls.warehouse_id
WHERE o.order_status <> 'Cancelled'
GROUP BY w.warehouse_id, w.warehouse_name
ORDER BY revenue DESC;
```

Challenge 4: Performance Tuning

1. Optimize the slowest queries using EXPLAIN

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT
  p.product_id,
  p.product_name,
  COUNT(oi.order_item_id) AS times_ordered,
  COALESCE(SUM(oi.quantity), 0) AS total_quantity
FROM products p
LEFT JOIN order_items oi ON oi.product_id = p.product_id
GROUP BY p.product_id, p.product_name
ORDER BY times_ordered DESC;
```

And for cohort query:

```
EXPLAIN (ANALYZE, BUFFERS)
WITH cohorts AS (
  SELECT c.customer_id, date_trunc('month', c.registration_date)::date AS cohort_month
  FROM customers c
),
activity AS (
  SELECT co.cohort_month, date_trunc('month', o.order_date)::date AS order_month,
       COUNT(DISTINCT o.customer_id) AS active_customers
  FROM cohorts co
  JOIN orders o ON o.customer_id = co.customer_id
  WHERE o.order_status <> 'Cancelled'
  GROUP BY co.cohort_month, date_trunc('month', o.order_date)::date
)
SELECT * FROM activity;
```

2. Create appropriate indexes

```
-- Inventory balancing + reorder totals
CREATE INDEX IF NOT EXISTS idx_inventory_product ON inventory(product_id);
CREATE INDEX IF NOT EXISTS idx_inventory_warehouse ON inventory(warehouse_id);
CREATE INDEX IF NOT EXISTS idx_inventory_product_warehouse ON inventory(product_id,
warehouse_id);


-- Cohorts + churn
CREATE INDEX IF NOT EXISTS idx_customers_registration_date ON customers(registration_date);


-- Revenue by warehouse + shipment attribution
CREATE INDEX IF NOT EXISTS idx_shipments_order_warehouse_date ON shipments(order_id,
warehouse_id, shipment_date);


-- Time-based analytics
CREATE INDEX IF NOT EXISTS idx_orders_order_date ON orders(order_date);


-- Order item analysis (pairs, discounts)
CREATE INDEX IF NOT EXISTS idx_order_items_order_product ON order_items(order_id,
product_id);
```

3. Rewrite a subquery using JOINs for better performance

```
SELECT
  p.product_id,
  p.product_name,
  COUNT(oi.order_item_id) AS times_ordered
FROM products p
LEFT JOIN order_items oi
  ON oi.product_id = p.product_id
GROUP BY p.product_id, p.product_name
HAVING COUNT(oi.order_item_id) = 0
ORDER BY p.product_name;
```